VERİ YAPILARI

🎓 Yüksek Lisans Günlük Notu — 2026-10-21

📚 Ders & Konu:

- [ ] VERİ YAPILARI LAB DERSİ TREE'LERE GEÇTİK

```
#recursive süreç
bir ağacın her node'unu root kabul edersek ihtiyacımız olan kısmı arayabiliriz
işte buna recursive deniyor
class Node:
     def __init__(self, data):
          self.data = data
          self.left= None
          self.right=None


class BST:
     def __init__(self):
          self.root =None

     def insert(self,data):
          self.root= self._insert(self.root,data)



     def _insert(self, node, data):
          if node is None:
               return Node(data)

          if data < node.data:
               node.left= self._insert(node.left, data)

          elif data > node.data:
               node.right = self._insert(node.right, data)
          return node

     #SEARCH KOMUTU

     def search(self,data):
          return self._search(self.root, data)

     def _search(self,node, data):
          if node is None:
               return False
```

```
        if node. data== data:
            return True
        if data <node.data:
            return self._search(node.left, data)

        else:
            return self._search(node.right, data)

    #DELETE
    hiç çocuğu yoksa direkt sil
    1 çocuğu varsa onun yerine koy
    2 çocuğu varsa sıkıntı

    def delete(self,data):
        self.root= self._delete(self.root, data)


    def_delete(self,node,data):
        if node is None:
            return None

        if data < node.data:
            node.left= self._delete(node.left, data)

        elif data> node.data:
            node.right = self._delete(node.right, data)
        else:
            if node.left is None and node.right is None:
                return None
            elif node.left is None:
                return node.right
            elif node.right is None:
                return node.left
            else:
                 successor= self._find_min(node.right)

                 node.data=successor.data
                 self._delete(node.right,successor.data)
    def_find_mind(self, node):
        while node.left is not None:
            node=node.left
        return node
    ROOT'UN YERİNİ BELİRLEYEN KELİMELER:
                  ROOT  LEFT   RİGHT
    PRE ÖN       RO      L      Rİ
    IN ARASINDA  L        RO    Rİ
    POST SONRASI  L       Rİ    RO
```

2 ÖDEV VERECEK

```
ŞU AN YAPMAKTA OLDUĞUMUZ DOSYANIN İÇERİSİNE DEF FİRSTH SEARCH
BİRDE UYGULAMA ÖDEVİ VERECEK
```

📖 Haftalık Ders Notları

- **Kaynak Adı:**
- **Önemli Bulgular / Alıntılar:**

TREE'ler NEDİR?

bir veri yapısıdır, non lineerdir yani sıralı olmadığından lineer değildir.

💡 Önemli Kavramlar & Konu Özeti

-

✍️ Uygulama, Kod (Lab)

İncelenen Araç / Protokol:

Komutlar & Betik / Konfigürasyon Notları:

Karşılaşılan Hata / Çözüm:

Takılan Noktalar & Araştırılacaklar (*Aksis Dökümanlar vs.*)

-

> **Sonraki Adım / Haftalık Hedef:**

- [ ]